Datastraw

DATASTRAW ASSESSMENT TEST

Hiring Assignment: Build a Support CRM System

Submission Deadline: 3-4 days

PROJECT SPECIFICATIONS

Must include: Full-stack web application (Database, API, Frontend)

Deliverables: Deployed web application, GitHub repository, and demo video (required)

Customer Support Ticketing CRM System

Build a fully functional web-based customer support management system that handles support tickets, customer data, order information, and team collaboration. The system should integrate a database, backend API, and a professional frontend. This assignment evaluates your ability to design end-to-end solutions, work with database schemas, build intuitive user interfaces, and deploy applications to production.

OVERVIEW

We believe the best way to learn is by building real things. This assignment is your opportunity to build a working tool from scratch, deploy it to production, and get a chance to intern at Datastraw Technologies.

Duration 2–3 days of focused work	**Scope** Full-stack web app (DB, API, Frontend)
Outcome Deployed app + GitHub + Demo Video	

WHAT YOU WILL BUILD

A web application for managing customer support tickets. Core functionality: Create tickets with customer info, search/filter, update status, and view details. This is a real system. Your code will run on actual servers. You will own the deployment and the product.

KEY FEATURES

Requirement: Build 4-5 core features listed below. See the 'Stand Out' section further down for how extra effort here is evaluated.

1. CREATE TICKETS • Customer name and email • Issue title and description • Auto-generated ticket ID & timestamp	**2. LIST ALL TICKETS** • Clean list view displaying ID, Name, Title, Status, and Date
3. SEARCH FUNCTIONALITY • Quick search across names, IDs, emails, and descriptions	**4. FILTER BY STATUS** • Filter by: Open, In Progress, Closed
5. VIEW & UPDATE TICKETS • Detailed view for each ticket • Update status, add notes/comments	

TECHNOLOGY STACK (YOUR CHOICE)

Choose the stack you are most comfortable with or want to learn. All choices are evaluated equally.

1. Python + FastAPI DB: SQLite Frontend: HTML + Tailwind/React Deploy: Railway.app	**2. Node.js + Express** DB: SQLite or MongoDB Frontend: React/Vue/HTML Deploy: Vercel/Render
3. Ruby on Rails DB: PostgreSQL Frontend: Rails views/React Deploy: Render.com	**4. Go** DB: SQLite or PostgreSQL Frontend: HTML/JS Deploy: Railway/Docker

Option 5: Any other modern framework or tools like supabase, vercel, etc is preferred. All modern stacks are acceptable if the product works well.

DATABASE DESIGN (KEEP IT SIMPLE)

Your database needs only 2 tables. Do not over-engineer the schema.

TICKETS TABLE	NOTES TABLE (OPTIONAL)
id (pk) ticket_id (unique, e.g., TKT-001) customer_name (text) customer_email (text) subject (text) description (text) status (Open/In Progress/Closed) created_at (timestamp) updated_at (timestamp)	id (pk) ticket_id (fk to tickets) note_text (text) created_at (timestamp)

API ENDPOINTS (SIMPLE REST)

Build these 4 endpoints—simple and sufficient:

- **POST /api/tickets** — Body: { customer_name, customer_email, subject, description } — Returns: { ticket_id, created_at }
- **GET /api/tickets** — Query params: ?status=Open&search=customer_name (Optional) — Returns: [{ ticket_id, customer_name, subject, status, created_at }]
- **GET /api/tickets/{ticket_id}** — Returns: { ticket_id, customer_name, customer_email, subject, description, status, notes }
- **PUT /api/tickets/{ticket_id}** — Body: { status, notes } — Returns: { success: true, updated_at }

FRONTEND REQUIREMENTS

Essential pages: • Home page with list of all tickets • Form to create a new ticket • Detail page for each ticket • Search bar (works as you type) & Filter	**Design & Tech:** • Keep it simple, clean, and usable. • Tailwind CSS, Bootstrap, or plain CSS. • Vanilla JS, React, Vue, or Svelte. • Mobile responsive recommended.

BONUS / STANDOUT (OPTIONAL)

The core features above will get you a passing submission. But we're not just evaluating whether it works, we're evaluating how you think.

If you have extra time, consider: what would make this genuinely useful for a real support team handling hundreds of tickets a day, across multiple channels, for multiple clients? What's missing from a bare-bones CRM that a real team would actually need?

You don't need to build something big or complex. One well-reasoned addition, clearly explained, is worth more than five shallow ones. In your submission email, briefly explain what you added, why you chose it, and what tradeoff you made.

This section is genuinely optional a clean, working core submission is a strong submission on its own.

CAN YOU USE AI TOOLS?

You should use AI tools ChatGPT, Claude, GitHub Copilot, etc. This is how real developers work.

- **What we expect:** You understand your code. You can explain how it works. You modified AI-generated code to make it yours.

- **What we do NOT expect:** "Generate the entire CRM for me" and submitting unchanged code. We will see this immediately.

DEPLOYMENT (REQUIRED)

Your application must be live on the internet. Recommended options:

- FastAPI → Railway.app
- Node.js → Vercel, Railway, or Render
- Rails → Render.com
- Go → Railway.app or Docker

Deployment Note: Do not spend hours on deployment. Use free, straightforward options. Getting your app live is the goal, not perfection.

WHAT TO SUBMIT

1. **Deployed Application (Required):** A public URL where evaluators can create tickets, search and filter, update tickets, and verify the app works. Make sure it is stable.
2. **GitHub Repository (Required):** Include all source code, README.md with setup instructions, .env.example, .gitignore, and a clean folder structure.
3. **Demo Video (Required):** A 3–5 min video showing the app working, a brief code walkthrough, tech choices explanation, and challenges solved.
4. **Submission Email:** Send links to the URL, Repo, and Video, plus 2–3 sentences on your approach.

EVALUATION CRITERIA

Criteria	Acceptable	Strong
Deployed & Working	App loads, features work	All features stable and polished
Code & API	Readable code, working endpoints	Well-structured, properly handled errors
Features & DB	2 features, stores data correctly	3+ features, clean schema/relationships
Frontend & Explanation	Functional UI, can explain choices	Professional UI, clear architecture understanding
Initiative	Meets core spec only	Adds a thoughtful extra from the Stand Out section, can explain the reasoning behind it.

> **Important:** We care more about shipped code than perfect code. A working application with good explanations is excellent. Perfect code that does not work is not.

WHAT WE ARE LOOKING FOR

✓ Can you take a requirement and build a solution?

- ✓ Can you learn new tools and frameworks quickly?
- ✓ Can you ship AI-generated code to production?
- ✓ Can you use AI tools to move faster and explain what you built?
- ✓ Do you think end-to-end (database → API → frontend)?

COMMON QUESTIONS

Q: I don't know Python/Node/Go.

A: Pick one and learn it. You have 2–3 days and AI tools. Get started.

Q: Can I use templates or starter code?

A: Yes. Use them wisely, but make sure you understand and modify the code.

Q: What if my code is not perfect?

A: Perfect is not the goal. Working is. Clean and readable is enough.

Q: What about authentication?

A: Basic auth is fine, or skip it for MVP. Keep it simple.

Q: Should I add fancy features?

A: Core features come first make sure those are solid. If you have time left, see the 'Stand Out' section above; thoughtful extras are noticed.

Q: What if I get stuck or run out of time?

A: Use AI tools, Stack Overflow, and docs. Submit what you completed and explain it clearly.

TIMELINE & DEADLINE

Start: Whenever you are ready.

Deadline: Submit within 3-4 days of starting.

Typical timeline: Most candidates complete in 2–3 days of focused work.

FINAL THOUGHTS

This assignment tests if you can learn by doing, ship real code, think end-to-end, and problem-solve. These are the skills that matter in real development.

Do not overthink it. Build something. Ship it. Show us. We are excited to see what you create.

SUBMISSION

Submit the project with a brief cover letter or message including:

- Your technical approach and architectural decisions
- Key features or aspects you are most proud of
- Challenges you faced and how you overcame them
- Improvements you would implement with additional time
- Public link to deployed Google Apps Script web application
- GitHub repository with complete source code
- Link to demo video on YouTube or similar platform

Please submit your completed assignment to ozair.shaikh@datastraw.in and put hr@datastraw.in in CC of the email.

Note: Please also attach your LinkedIn profile link in the email. Kindly make sure the link is working.